

	Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores Computação Distribuída (Época Normal) MEIC, MEIM	
Nº	Nome:	08/01/2025

EXAME ÉPOCA NORMAL (1ª PARTE, SEM CONSULTA, DURAÇÃO: 45 min., 11 valores)

Nas questões de 1 a 5 assinale as afirmações como verdadeira (V), falsa (F) ou não responde (→). Uma opção assinalada corretamente soma 0,25 val. e incorretamente desconta 0,125 val. (50% da cotação da alínea).

1. [1 val.] Sobre características dos sistemas distribuídos

☐	<i>Speedup</i> é um número que indica a capacidade de processamento que um sistema é capaz de realizar num determinado tempo, por exemplo, número de pedidos por segundo
☐	<i>Speedup</i> é um número que indica a melhoria do tempo de execução de uma tarefa, quando executada em dois sistemas similares com configurações ou recursos diferentes
☐	A escalabilidade horizontal (<i>scaling out</i>) consiste em adicionar réplicas, por exemplo mais servidores, para executar a mesma aplicação, fazendo assim equilíbrio de carga (<i>load balancing</i>)
☐	Um sistema com replicação de dados em vários servidores requer a garantia de consistência do estado global do sistema, usando algoritmos de coordenação e consenso, permitindo assim maior escalabilidade

2. [1 val.] Considere a operação gRPC: `rpc upload(stream Block) returns (ACK);` com as mensagens: `message Block{ bytes data = 1; }` `message ACK{ boolean ack = 1; }`

☐	Na aplicação cliente faz sentido chamar a operação usando um <i>stub</i> não bloqueante, mas também pode ser chamada com um <i>stub</i> bloqueante, retornando um <i>Iterator</i> sobre um <i>stream</i> com blocos de dados
☐	A implementação no servidor retorna um objeto <code>StreamObserver<Block></code> à aplicação cliente para que esta envie dados para o servidor como uma sequência de blocos (<i>byte array</i>)
☐	Cada bloco de dados (campo <i>data</i> no tipo <code>Block</code>) é enviado chamando o método <code>onNext(Block bloco)</code> , de um objeto <code>StreamObserver<Block></code> devolvido à aplicação cliente pelo servidor. A sequência de dados termina quando a aplicação cliente chamar o método <code>onCompleted()</code>
☐	A implementação no servidor devolve, por cada bloco de dados, um objeto do tipo <code>ACK</code>

3. [1 val.] Sobre o *middleware* RabbitMQ

☐	É garantido que as mensagens são sempre consumidas pela mesma ordem em que foram publicadas
☐	Um consumidor pode estar associado e receber mensagens de múltiplas <i>queues</i>
☐	Um consumidor de uma <i>queue</i> pode rejeitar (<i>negative acknowledge</i>) uma mensagem individual sem afetar a receção de outras mensagens dessa mesma <i>queue</i>
☐	O processamento de mensagens de uma <i>queue</i> por múltiplos consumidores (padrão <i>work-queue</i>) não é possível se a <i>queue</i> estiver associada a um <i>exchange</i> do tipo <i>FANOUT</i>

4. [1 val.] Sobre algoritmos de exclusão mútua, eleição e consensos

☐	No algoritmo de exclusão mútua <i>Ricart-Agrawala</i> , um processo, antes de entrar na secção crítica, tem de receber a permissão de todos os outros processos
☐	No algoritmo de eleição <i>Bully</i> , quando se deteta que o coordenador falhou, só o processo que tem o maior identificador é que inicia a eleição, garantindo assim que será o novo coordenador
☐	No algoritmo de consenso <i>Raft</i> , nos diferentes servidores as entradas do <i>Log</i> com o mesmo índice e termo (<i>term</i>) têm sempre o mesmo valor
☐	No algoritmo de consenso <i>Two-Phase Commit</i> , a falha do coordenador, após enviar <i>prepare</i> (<i>vote request</i>) e antes de enviar <i>commit</i> (<i>decision phase</i>), provoca o bloqueio dos participantes indefinidamente

5. [1 val.] Sobre Docker *containers*

☐	Para executar um novo <i>container</i> temos sempre de definir um novo <i>Dockerfile</i>
☐	No comando <code>docker run</code> a opção <code>-p</code> permite redirecionar tráfego de portos <i>tcp/ip</i> do <i>host</i> para portos do <i>guest</i> (<i>container</i>)
☐	No comando <code>docker run</code> a opção <code>-v</code> permite partilhar ficheiros entre o sistema de ficheiros do <i>host</i> e o sistema de ficheiros do <i>guest</i> (<i>container</i>)
☐	A execução de um <i>container</i> é sempre realizada a partir de uma imagem (<i>docker image</i>)

6. [2 val.] Considere um sistema distribuído, constituído por 3 participantes (**p0**, **p1**, **p2**), que utiliza comunicação por *multicast* fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Quando **p0 envia a mensagem** com o vetor **[2,3,4]**, justifique o que acontece na receção da mensagem nos participantes p1 e p2 e indique como ficam os relógios locais, considerando:
- a) **p1** tem o seu vetor local **p1=[1,4,3]**;
 - b) **p2** tem o seu vetor local **p2=[0,4,4]**;

7. [2 val.] Considere que existe um conjunto de servidores gRPC que aceitam pedidos dos clientes para calcular se um número é ou não primo. Quando o servidor já conhece a resposta relativa a um número devolve-a de imediato ao cliente, se não conhecer a resposta processa o número para ver se ele é ou não primo. Para otimizar o conhecimento global de todos os servidores sobre números primos, um programador teve a ideia de tirar partido da comunicação por grupos com *multicast*, usando o Spread Toolkit. Indique em frases simples as vantagens e desvantagens da ideia do programador.

8. [2 val.] O Docker suporta o conceito de *volume* que permite que todos os *containers* em execução num mesmo *host* possam partilhar ficheiros de dados, tanto de *input* como de *output*. No entanto, num cenário com múltiplos *host*, por exemplo múltiplas máquinas virtuais (VM), não é possível aos *containers*, em execução nessas várias VM, partilharem ficheiros diretamente recorrendo a *volumes* Docker. Tendo como referência os *middleware* estudados, proponha uma solução para ultrapassar essa limitação, isto é, possibilitar a partilha de ficheiros entre múltiplos *containers* executados em múltiplas VM.